of code and models into a shared repository while maintaining strict version control. CI covers:

- *Automated code and model integration:* Merges code changes and ML models into a shared repository seamlessly.
- *Version control for code, data and models:* Utilises tools like Git, DVC or MLflow to track changes.
- *Automated testing of ML pipelines*: Implements unit tests, integration tests and validation tests for data and models.
- *Build reproducibility:* Ensures that builds can be replicated consistently across environments.
- *Enhanced collaboration*: Facilitates teamwork between data scientists, engineers and stakeholders.

Continuous deployment (CD) in MLOps: This moves models safely into production using strategies like blue-green or canary deployments to minimise risk. CD covers:

- *Automated model deployment:* Deploys machine learning models to production environments automatically.
- *Infrastructure as Code (IaC):* Uses Terraform or Kubernetes for scalable, reproducible deployments.
- *Blue-green and canary deployments:* Implements strategies for safe and incremental model releases.
- *Rollback mechanisms:* Enables quick rollback to previous model versions in case of issues.
- *Seamless integration with CI pipelines:* Ensures smooth transition from integration to deployment phases.

Continuous training (CT) in MLOps: This regularly updates models with fresh data to maintain their predictive power automatically. It covers:

- *Automated retraining of pipelines:* Schedules and triggers model retraining based on data updates or performance metrics.
- *Data versioning and management:* Tracks and manages datasets using tools like DVC or Delta Lake.
- *Hyperparameter tuning and experimentation:* Automates optimal parameter search with frameworks like Optuna or Hyperopt.
- *Scalable training infrastructure:* Leverages cloud resources or distributed computing for efficient training.
- *Model validation and testing:* Ensures new models meet performance and quality standards before deployment.

Continuous monitoring (CM) in MLOps: This tracks real-time metrics such as accuracy and latency using tools to identify issues immediately. It covers:

- *Real-time model performance monitoring:* Tracks metrics using tools like Prometheus or Grafana.
- *Data and model drift detection:* Identifies changes in data distribution or model performance.
- *Automated alerts and notifications:* Sets up alerts for anomalies or performance degradation.
- *Logging and auditing:* Maintains logs for compliance and troubleshooting.
- *Feedback loops for model improvement:* Uses monitoring insights for retraining and deployment cycles.

Table 1: DevOps and MLOps – a comparison

Parameter	DevOps	MLOps
Purpose	General software development and IT operations	Machine learning model development, deployment and monitoring
Collaboration	Developers, IT operations and security teams	Data scientists, ML engineers, DevOps engineers and analysts
Roles	DevOps engineers, system admins, software developers	MLOps engineers, data scientists, machine learning engineers
Key tools	Jenkins, Docker, Kubernetes, Ansible, Terraform	ML flow, DVC, Kubeflow, TFX, Airflow
Main component	Code	Code + Data + Model
Testing	Unit and integration tests	Back testing, drift detection, and bias audits
Lifecycle management	Focuses on app code lifecycle and infrastructure	Focuses on data, model lifecycle and continuous retraining
Primary focus	Code delivery, system automation and reliability	Data versioning, model reproducibility and performance
Deployment	Single software version	A/B testing and shadow deployments
Maintenance	Bug fixes	Retraining and fine-tuning
Performance monitoring	Primarily monitors system and application performance	Monitors model drift, accuracy and impact on real world data
Scalability	Infrastructure scaling for application needs	Scaling ML pipelines and model deployment

MLOps lifecycle

The process of building and maintaining an ML model is divided into four key functional areas.